

The
University of
Faisalabad

ARTIFICIAL INTELLIGENCE

LAB MANUAL

OPERATING SYSTEM

Name : Ayesha Imran

Student ID : 2023-BS-AI-011

Submitted to : Mam Irsha

LAB NO	LAB NAME	PAGE NO
01	Introduction to Operating System and CMD	03
02	Basic CMD Commands for Directory Management	05
03	Creating Folders Using CMD	07
04	Doing Tasks Using CMD, Folder Management Using DOS	10
05	Advanced File and Directory Management in CMD	12
06	Managing File Operations Using CMD	16
07	Internal and External Commands	19
08	First Come First Serve (FCFS)	31
09	Shortest Job First (SJF) Scheduling Algorithm Non-Primitive	38

LAB NO 01

INTRODUCTION TO OPERATING SYSTEM AND CMD

➤ **OPERATING SYSTEM:**

An Operating System (OS) is a software program that acts as the intermediary between a computer user and its hardware, managing core functions like file management, memory allocation, and running applications, essentially providing a platform for users to interact with the computer smoothly; basic commands within an OS allow users to perform simple tasks like creating files, navigating directories, and deleting data through text-based instructions in a command prompt or terminal window.

➤ **KEY FUNCTIONS OF OS:**

1. **File Management:**

Creating, deleting, renaming, and moving files within a hierarchical directory structure.

2. **Process Management:**

Managing the execution of multiple programs simultaneously, allocating CPU time efficiently.

3. **Memory Management:**

Allocating and managing system memory for different programs.

4. **Input/Output Management:**

Handling interactions between the computer and peripherals like keyboard, mouse, and monitor.

5. **Device Management:**

Recognizing and managing various hardware devices connected to the computer.

➤ **CMD PROMPT:**

Command Prompt, or CMD, is a text-based user interface program that allows users to run commands and troubleshoot Windows operating systems. It's also known as cmd.exe.

What can you do with CMD?

1. Run batch files

2. Perform advanced administrative functions
3. Troubleshoot and solve Windows issues
4. Shut down or restart computers
5. Create, delete, and format partitions

BASIC CMD COMMANDS:

1. **cd**

Changes your current directory.

2. **cd /**

Navigates to the root directory.

3. **dir**

Displays a list of files and subdirectories in a directory.

4. **mkdir (or md)**

Creates a new directory.

5. **rmdir (or rd)**

Removes a directory.

6. **rename (or ren)**

Renames a file. For example, rename file.txt newfile.txt changes the file's name to "newfile.txt"

LAB NO 02

BASIC CMD COMMANDS FOR DIRECTORY MANAGEMENT

Creating new folder :

```
OS Select Command Prompt
Microsoft Windows [Version 10.0.19045.5371]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Ayaan>cd desktop

C:\Users\Ayaan\Desktop>md operatingsystem
```

Output:



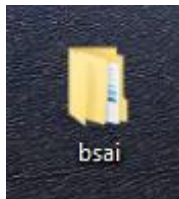
Rename folder:

```
C:\Users\Ayaan\Desktop>ren operatingsystem folder

C:\Users\Ayaan\Desktop>ren new bsai
The system cannot find the file specified.

C:\Users\Ayaan\Desktop>ren folder bsai
```

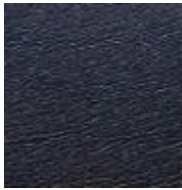
Output:



Remove folder:

```
C:\Users\Ayaan\Desktop>rd bsai
```

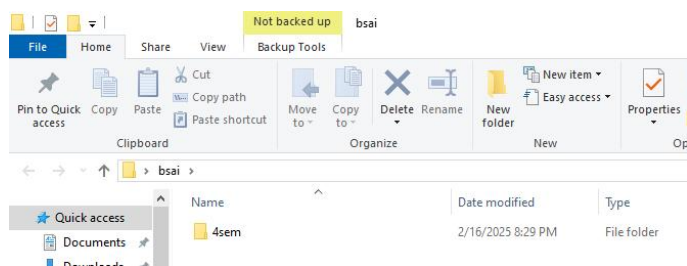
Output:



Making subfolders:

```
C:\Users\Ayaan\Desktop>md bsai\4sem\os
```

Output:



What if ?

```
C:\Users\Ayaan\Desktop>md bsai\4sem \os
A subdirectory or file \os already exists.
Error occurred while processing: \os.
```

LAB NO 3

CREATING FOLDERS USING CMD

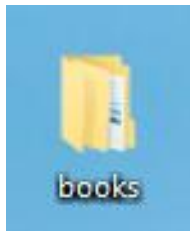
Making one folder “ books”

```
Command Prompt
Microsoft Windows [Version 10.0.19045.5487]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Ayaan>cd desktop

C:\Users\Ayaan\Desktop>md books
```

Output:

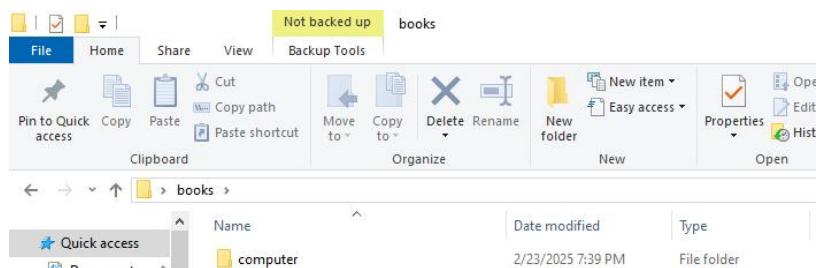


Making sub-folder “ computer “ and making sub folders “ trigonometry ” and “calculus” In computer folder

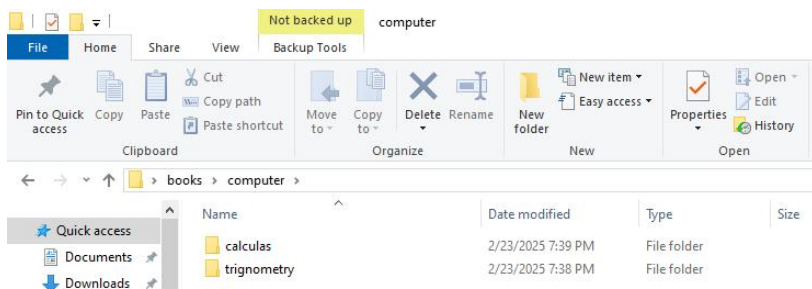
```
C:\Users\Ayaan\Desktop>md books\computer\trigonometry

C:\Users\Ayaan\Desktop>md books\computer\calculus
```

Output no 1:



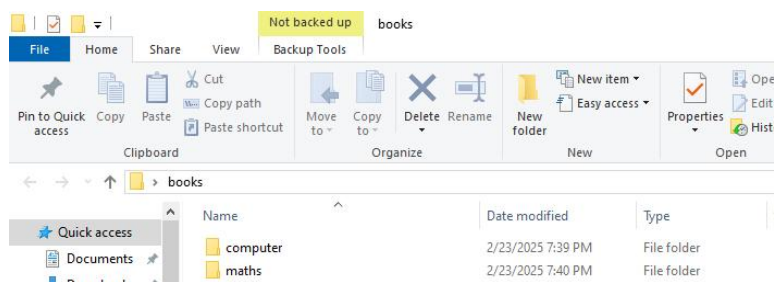
Output no 2:



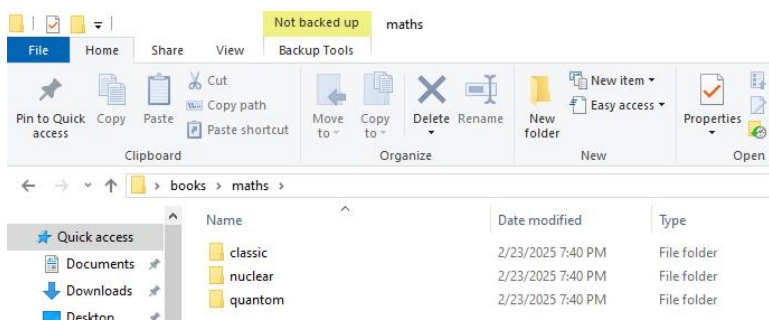
Making subfolder “ maths“ and making subfolders “ quantum” And “classic” And “ nuclear” In maths folder

```
C:\Users\Ayaan\Desktop>md books\maths\quantom  
C:\Users\Ayaan\Desktop>md books\maths\classic  
C:\Users\Ayaan\Desktop>md books\maths\nuclear
```

Output no 1:



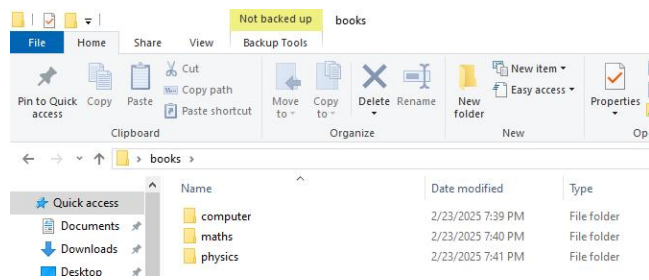
Output no 2:



Making subfolder “ physics “

```
C:\Users\Ayaan\Desktop>md books\physics
```

Output:



LAB NO 4

DOING TASKS USING CMD, FOLDER MANAGEMENT USING DOS

Using Tree Command:

```
Command Prompt
Microsoft Windows [Version 10.0.19045.5487]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Ayaan>cd desktop

C:\Users\Ayaan\Desktop>md ayeshaaa

C:\Users\Ayaan\Desktop>md ayeshaaa\rollno

C:\Users\Ayaan\Desktop>md ayeshaaa\semsno

C:\Users\Ayaan\Desktop>md ayeshaaa\rollno\a1

C:\Users\Ayaan\Desktop>md ayeshaaa\semsno\b1

C:\Users\Ayaan\Desktop>tree "ayeshaaa"
Folder PATH listing
Volume serial number is 000000C1 3C7A:8BDA
C:\USERS\AYAAN\DESKTOP\AYESHAAA
├── rollno
│   └── a1
└── semsno
    └── b1

C:\Users\Ayaan\Desktop>_
```

Hiding folder:

```
C:\Users\Ayaan\Desktop>attrib +h "ayeshaaa"
```

Unhiding folder:

```
C:\Users\Ayaan\Desktop>attrib -h "ayeshaaa"
```

Hiding subfolder:

```
C:\Users\Ayaan\Desktop>attrib +h "ayeshaaa\rollno"
```

Folder Management using DOS :

 Command Prompt

```
Microsoft Windows [Version 10.0.19045.5487]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Ayaan>cd desktop

C:\Users\Ayaan\Desktop>md TestFolder\SubFolder1

C:\Users\Ayaan\Desktop>attrib +h "TestFolder\SubFolder1"

C:\Users\Ayaan\Desktop>attrib -h "TestFolder\SubFolder1"
```

Removing a Folder with Sub folders:

 Command Prompt

```
Microsoft Windows [Version 10.0.19045.5487]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Ayaan>cd desktop

C:\Users\Ayaan\Desktop>md DeleteMe\Sub1

C:\Users\Ayaan\Desktop>md DeleteMe\Sub2

C:\Users\Ayaan\Desktop>rd /s /q "DeleteMe"
```

Execute the following commands on Desktop

- make the D drive as current working drive

```
C:\Users\Ayaan\Desktop>D:

D:\>_
```

- tree (on any folder at Desktop)

```
D:\>md folder1\subfolder11\subfolder12

D:\>md folder1\subfolder21\subfolder22

D:\>tree "folder1"
Folder PATH listing for volume New Volume
Volume serial number is 00000096 10C8:2037
D:\FOLDER1
├──subfolder11
│   └──subfolder12
├──subfolder21
│   └──subfolder22
```

LAB 5**ADVANCED FILE AND DIRECTORY MANAGEMENT IN CMD****Making main folder:**

```
Command Prompt
Microsoft Windows [Version 10.0.19045.5487]
(c) Microsoft Corporation. All rights reserved.

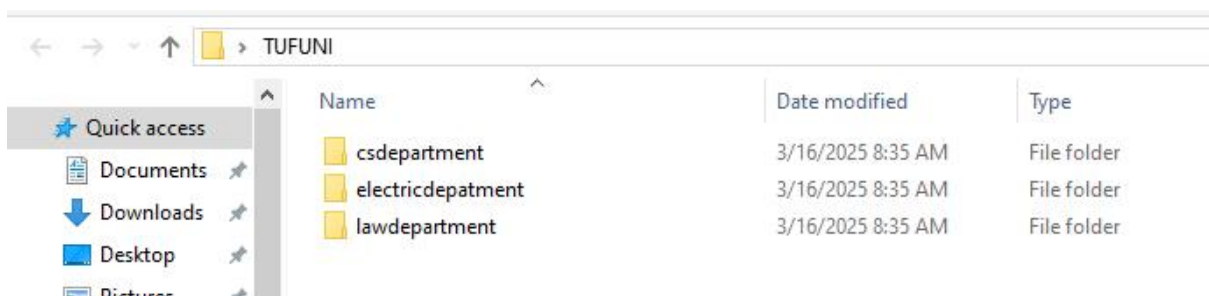
C:\Users\Ayaan>cd desktop

C:\Users\Ayaan\Desktop>md TUFUNI

C:\Users\Ayaan\Desktop>
```

Output:**Making subfolders:**

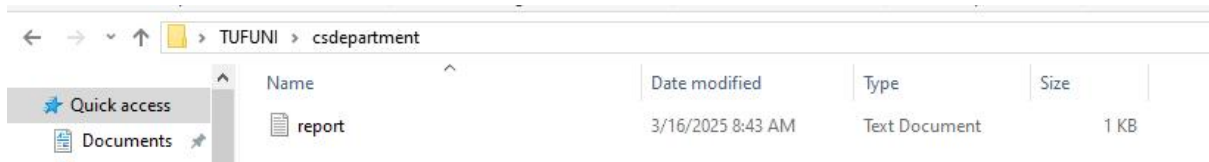
```
C:\Users\Ayaan\Desktop>md TUFUNI\csdepartment
C:\Users\Ayaan\Desktop>md TUFUNI\lawdepartment
C:\Users\Ayaan\Desktop>md TUFUNI\electricdepatment
```

Output:

Making a file “report” in csdepartment folder:

```
C:\Users\Ayaan\Desktop>echo This is my document. >TUFUNI\csdepartment\report.txt
```

Output:



The screenshot shows a Windows File Explorer window with the address bar set to 'TUFUNI > csdepartment'. The left sidebar shows 'Quick access' with 'Documents' selected. The main pane displays a table with the following content:

Name	Date modified	Type	Size
report	3/16/2025 8:43 AM	Text Document	1 KB

Tree structure:

```
C:\Users\Ayaan\Desktop>tree "TUFUNI"
Folder PATH listing
Volume serial number is 00000031 3C7A:8BDA
C:\USERS\AYAAN\DESKTOP\TUFUNI
|_ csdepartment
|_ electricdepatment
|_ lawdepartment
```

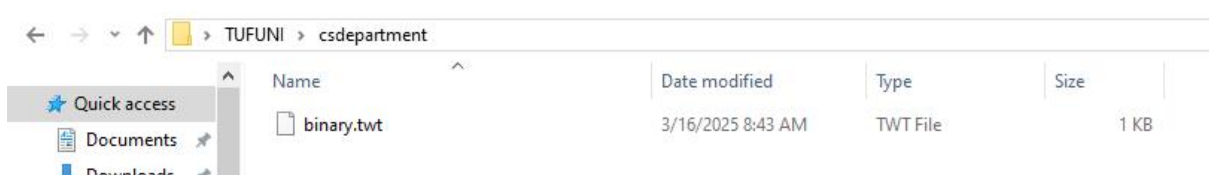
Tree structure to show file in subfolder:

```
C:\Users\Ayaan\Desktop>tree TUFUNI /f
Folder PATH listing
Volume serial number is 00000033 3C7A:8BDA
C:\USERS\AYAAN\DESKTOP\TUFUNI
|_ csdepartment
|   |_ report.txt
|_ electricdepatment
|_ lawdepartment
```

Renaming “report” file as “binary”:

```
C:\Users\Ayaan\Desktop>ren TUFUNI\csdepartment\report.txt binary.twt
```

Output:



The screenshot shows a Windows File Explorer window with the address bar set to 'TUFUNI > csdepartment'. The left sidebar shows 'Quick access' with 'Downloads' selected. The main pane displays a table with the following content:

Name	Date modified	Type	Size
binary.twt	3/16/2025 8:43 AM	TWT File	1 KB

Move “binary” file from csdepartment folder to lawdepartment folder:

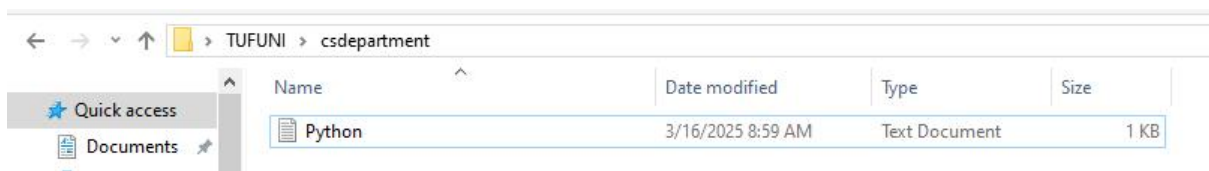
```
C:\Users\Ayaan\Desktop>move TUFUNI\csdepartment\binary.twt TUFUNI\lawdepartment\  
1 file(s) moved.
```

Erase folder “lawdepartment”:

```
C:\Users\Ayaan\Desktop>erase TUFUNI\lawdepartment\  
C:\Users\Ayaan\Desktop\TUFUNI\lawdepartment\*, Are you sure (Y/N)? y
```

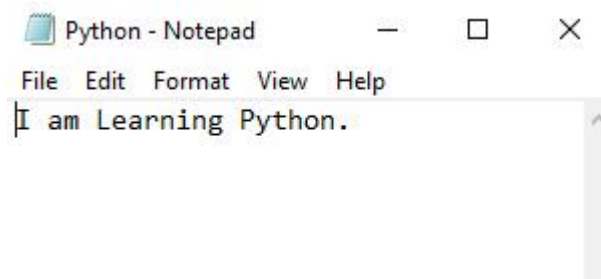
Writing a sentence in txt file:

```
C:\Users\Ayaan\Desktop>echo I am Learning Python. >TUFUNI\csdepartment\Python.txt
```



Show the sentence on command prompt:

```
C:\Users\Ayaan\Desktop>type TUFUNI\csdepartment\Python.txt  
I am Learning Python.
```



Setting date:

```
C:\Users\Ayaan\Desktop>date  
The current date is: Sun 03/16/2025  
Enter the new date: (mm-dd-yy) 05/03/2025
```


LAB NO 6

MANAGING FILE OPERATIONS USING CMD

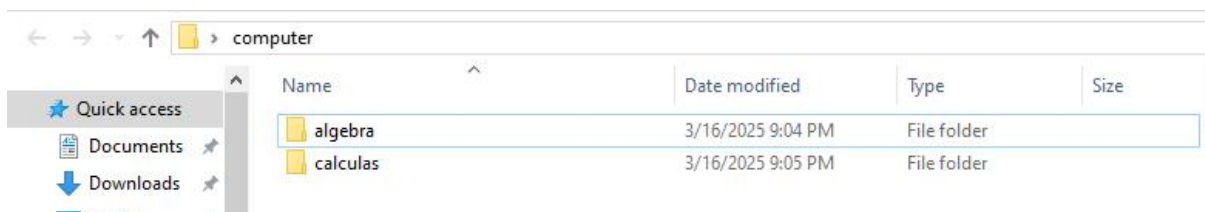
Making Folders and Subfolders

```

CA. Command Prompt
Microsoft Windows [Version 10.0.19045.5487]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Ayaan>cd desktop
C:\Users\Ayaan\Desktop>md computer\calculus computer\algebra
  
```

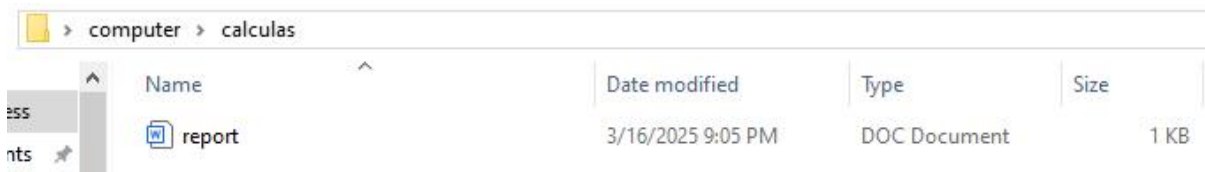
Output:



Name	Date modified	Type	Size
algebra	3/16/2025 9:04 PM	File folder	
calculus	3/16/2025 9:05 PM	File folder	

```

C:\Users\Ayaan\Desktop>echo MY NAME IS AYESHA. >computer\calculus\report.doc
  
```



Name	Date modified	Type	Size
report	3/16/2025 9:05 PM	DOC Document	1 KB

```

C:\Users\Ayaan\Desktop>echo MY NAME IS DOC. >computer\algebra\binary.doc
  
```

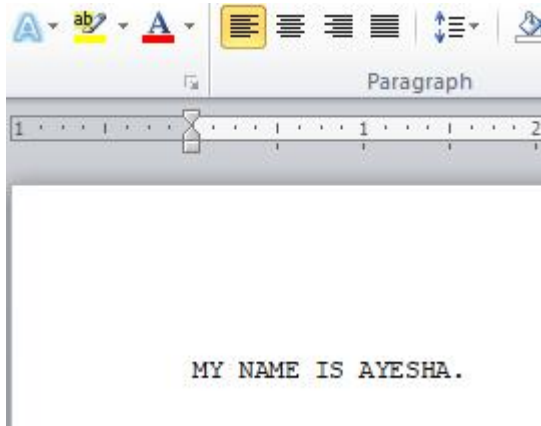


Name	Date modified	Type
binary	3/16/2025 9:07 PM	DOC Docume

```

C:\Users\Ayaan\Desktop>copy computer\calculus\report.doc computer\algebra\binary.doc
Overwrite computer\algebra\binary.doc? (Yes/No/All): yes
1 file(s) copied.
  
```

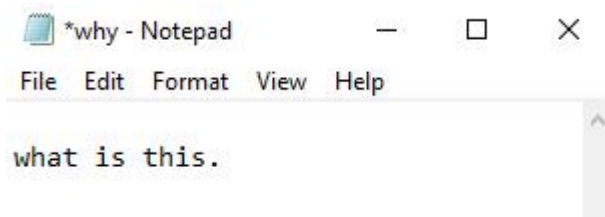
```
C:\Users\Ayaan\Desktop>start winword computer\algebra\binary.doc
```



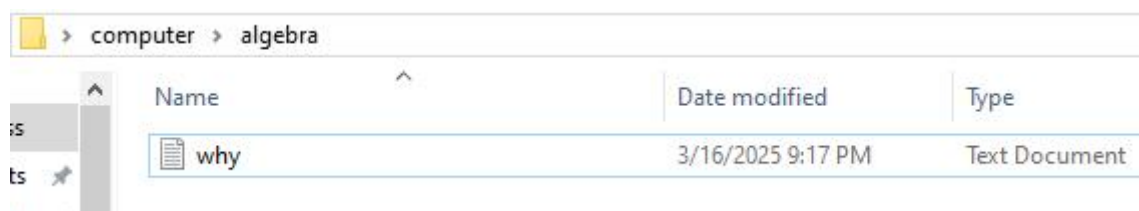
```
C:\Users\Ayaan\Desktop>echo what is this. >computer\algebra\why.txt
```



```
C:\Users\Ayaan\Desktop>start notepad computer\algebra\why.txt
```



```
C:\Users\Ayaan\Desktop>del computer\algebra\binary.doc
```



LAB NO 07

Internal and External Commands

List of Commands to Implement:

1. **Date:**
2. **Time:**
3. **Ver:**
4. **Winver:**
5. **Cls:**
6. **Ipconfig:**
7. **Netsh wlan show profile name='____' keyclear:**
8. **Logoff:**
9. **Start:**
10. **Shutdown:**
11. **Echo:**
12. **Help:**
13. **Move:**
14. **Copy:**
15. **Tree:**
16. **Rd:**
17. **Ren:**
18. **Md:**
19. **Cd:**
20. **Exit:**
21. **Del:**

With Description:

Descriptions of the CMD commands you listed:

1. Date

Description: Displays or sets the system date. If used without parameters, it shows the current date and prompts you to enter a new one.

2. Time

Description: Displays or sets the system time. If used without parameters, it shows the current time and prompts you to enter a new one.

3. Ver

Description: Displays the current version of the Windows operating system you're using.

4. Winver

Description: Displays the version of Windows you are running, along with other system information like the build number.

5. Cls

Description: Clears the Command Prompt screen, removing all previously displayed text.

6. Ipconfig

Description: Displays the current network configuration, including the IP address, subnet mask, and default gateway of all network adapters on the machine.

7. Netsh wlan show profile name='____' key=clear

Description: Shows the Wi-Fi profile of a specific network (replace '____' with the actual Wi-Fi network name). The key=clear argument reveals the saved Wi-Fi password if available.

8. Logoff

Description: Logs off the current user session, effectively ending the user's session without shutting down the computer

9. Start

Description: Launches a program, opens a file, or starts a new instance of a command prompt or other application. For example, start notepad opens Notepad.

10. Shutdown

Description: Shuts down, restarts, or logs off the computer. You can specify various options, like shutdown /s /f to force a shutdown.

11. Echo

Description: Displays a line of text or toggles command echoing on/off. For example, echo Hello displays "Hello" on the screen.

12. Help

Description: Displays a list of available commands in the Command Prompt or provides detailed help for a specific command when used with a command name (e.g., help copy).

13. Move

Description: Moves files or directories from one location to another. For example, move file.txt D:\backup\ moves file.txt to the D:\backup\ folder.

14. Copy

Description: Copies files or directories from one location to another. For example, copy file.txt D:\backup\ copies file.txt to the D:\backup\ folder.

15. Tree

Description: Displays a graphical directory tree of the current folder or a specified path, showing the structure of subdirectories.

16. Rd

Description: Removes a directory. The directory must be empty before it can be removed. For example, rd foldername deletes foldername.

17. Ren

Description: Renames a file or directory. For example, ren oldname.txt newname.txt renames a file from oldname.txt to newname.txt.

18. Md

Description: Creates a new directory (folder). For example, md newfolder creates a folder named newfolder.

19. Cd

Description: Changes the current directory (folder). For example, cd C:\Users changes the current directory to C:\Users.

20. Exit

Description: Exits the Command Prompt or closes the current batch script.

21. Del

Description: Deletes one or more files. For example, del file.txt deletes the file file.txt. Be cautious, as the deleted files cannot be easily recovered.

With Syntax:

1. Date

Syntax:

date

- o Displays the current system date.
- o To set the date, just enter the new date when prompted, in MM-DD-YYYY format.

2. Time

Syntax:

time

- o Displays the current system time.
- o To set the time, just enter the new time when prompted, in HH:MM:SS format.

3. Ver

Syntax:

ver

- o Displays the current version of the Windows operating system.

4. Winver

Syntax:

winver

- o Displays the detailed Windows version, including the build number.

5. Cls

Syntax:

cls

- o Clears the screen in Command Prompt.

6. Ipconfig

Syntax:

ipconfig

- o Displays the IP configuration of the system.

For detailed information:

- o ipconfig /all
- o Lists all network interfaces and their details (including MAC address, DNS, etc.).

7. Netsh wlan show profile name="" ____ " key=clear

Syntax:

netsh wlan show profile name="SSID" key=clear

- o Replace "SSID" with the actual Wi-Fi profile name (without the quotes).
- o Reveals saved Wi-Fi credentials (including the password) for the specified network.

8. Logoff

Syntax:

logoff

- o Logs off the current user from the system.

9. Start

Syntax:

start [program_name or file_path]

- o Example:
- o start notepad
- o Starts the specified program or opens the specified file.

10. Shutdown

Syntax:

shutdown /s /f /t [seconds]

- o /s: Shutdown the system.
- o /f: Forces running applications to close.
- o /t [seconds]: Set a timer for shutdown in seconds (e.g., /t 60 to shutdown after 60 seconds).
- o For example, to shutdown immediately:
- o shutdown /s /f /t 0

11. Echo

Syntax:

echo [message]

- o Displays a message.
- o Example:

- o echo Hello, World!
- o To toggle command echoing on or off:
- o echo on
- o echo off

12. Help

Syntax:

help

- o Displays a list of available commands in the Command Prompt.

o For a specific command:

o help [command_name]

o Example:

o help copy

13. Move

Syntax:

move [source] [destination]

- o Moves a file or directory from one location to another.

o Example:

o move file.txt D:\backup\

14. Copy

Syntax:

copy [source] [destination]

- o Copies a file from one location to another.

o Example:

o copy file.txt D:\backup\

15. Tree

Syntax:

tree [drive:\path]

- o Displays the directory tree of the specified path. If no path is provided, it shows the structure of the current directory.

o Example:

o tree C:\

16. Rd

Syntax:

rd [directory_name]

- o Removes an empty directory.

o Example:

o rd foldername

17. Ren

Syntax:

ren [old_name] [new_name]

- o Renames a file or directory.

o Example:

o ren oldname.txt newname.txt

18. Md

Syntax:

md [directory_name]

- o Creates a new directory.
- o Example:
- o md newfolder

19. Cd

Syntax:

cd [path]

- o Changes the current directory.
- o Example:
- o cd C:\Users

20. Exit

Syntax:

exit

- o Exits the Command Prompt or ends a batch script.

21. Del

Syntax:

del [filename]

- o Deletes a file.
- o Example:
- o del file.txt

Task to Execute:

1. View System Information

Task:

Use ver and winver to view the version and build number of your operating system.
Check the system date and time using date and time.

Commands to Use:

ver

CA. Command Prompt

```
Microsoft Windows [Version 10.0.19045.5608]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Ayaan>ver

Microsoft Windows [Version 10.0.19045.5608]
```

Winver

```
C:\Users\Ayaan>winver
```



Date:

```
C:\Users\Ayaan>date
The current date is: Sun 04/20/2025
Enter the new date: (mm-dd-yy) 05/20/2025
A required privilege is not held by the client.
```

time

```
C:\Users\Ayaan>time
The current time is: 21:57:20.48
Enter the new time: 12:12:14.11
A required privilege is not held by the client.
```

2. Network Configuration

Task:

Use ipconfig to view your system's network configuration.

Use netsh wlan show profile name="YourWiFiNetwork" key=clear to display saved Wi-Fi passwords (if applicable).

Commands to Use:

ipconfig

```

C:\Users\Ayaan> ipconfig

Windows IP Configuration

Ethernet adapter Ethernet:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Unknown adapter Local Area Connection:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 9:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 10:

    Media State . . . . . : Media disconnected
    Connection-specific DNS Suffix  . :

Wireless LAN adapter Wi-Fi:

    Connection-specific DNS Suffix  . : mshome.net
  
```

netsh wlan show profile name="YourWiFiNetwork" key=clear

```

C:\Users\Ayaan> netsh wlan show profile name=HI key=clear

Profile HI on interface Wi-Fi:
=====

Applied: All User Profile

Profile information
-----
    Version           : 1
    Type              : Wireless LAN
    Name              : HI
    Control options    :
        Connection mode : Connect automatically
        Network broadcast : Connect only if this network is broadcasting
        AutoSwitch       : Do not switch to other networks
        MAC Randomization : Disabled

Connectivity settings
-----
    Number of SSIDs    : 1
    SSID name         : "HI"
    Network type       : Infrastructure
    Radio type         : [ Any Radio Type ]
    Vendor extension   : Not present

Security settings
-----
    Authentication     : WPA2-Personal
  
```

3. Create and Manage Files and Folders

Task:

- Create a new directory, then navigate into it.
- Create a text file, copy it to another folder, and rename it.
- Delete the text file and the directory afterward.

Commands to Use:

md newFolder

```
C:\Users\Ayaan>md ayesha111
```

cd newFolder

```
C:\Users\Ayaan>cd desktop
```

echo Hello, World! > file.txt

```
C:\Users\Ayaan\Desktop>echo Hello, World!  
Hello, World!
```

copy file.txt D:\backup\

```
C:\Users\Ayaan\Desktop>copy "C:\Users\Ayaan\Desktop\2023-bsai-011.pptx" D:\  
Overwrite D:\2023-bsai-011.pptx? (Yes/No/All): yes  
1 file(s) copied.
```

ren file.txt newfile.txt

```
C:\Users\Ayaan\Desktop>ren 2023-bsai-011.pptx esppe.pptx
```



del newfile.txt

```
C:\Users\Ayaan\Desktop>del esppe.pptx
```

rd .. (to remove the directory)

```
C:\Users\Ayaan\Desktop>rd ayesha111
```

4. File Backup Using Copy and Move

Task:

- Create a new folder and copy some files into it.
- Move the files to another location and then remove the original folder.

Commands to Use:

md backup

```
C:\Users\Ayaan\Desktop>md backup
```

copy *.txt backup\

```
C:\Users\Ayaan\Desktop>copy 2023-bsai-011.pptx backup\  
1 file(s) copied.
```

```
move backup\*.txt D:\backup\  
C:\Users\Ayaan\Desktop>move backup\2023-bsai-011.pptx D:\  
The file exists.  
0 file(s) moved.
```

```
rd backup  
C:\Users\Ayaan\Desktop>rd backup
```

5. View Directory Structure

Task:

Use tree to visualize the directory structure of a drive (e.g., C:).

Output the directory structure to a text file.

Commands to Use:

tree C:\ > directory_structure.txt

```
Command Prompt  
C:\Users\Ayaan> tree C:\  
Folder PATH listing  
Volume serial number is 00000089 3C7A:8BDA  
C:\  
--oraclexe  
--|--app  
--|--|--oracle  
--|--|--|--flash_recovery_area  
--|--|--|--|--XE  
--|--|--|--|--ONLINELOG  
--|--|--|--product  
--|--|--|--10.2.0  
--|--|--|--|--server  
--|--|--|--|--|--BIN  
--|--|--|--|--|--database  
--|--|--|--|--|--NETWORK  
--|--|--|--|--|--|--log  
--|--|--|--|--|--|--mesg  
--|--|--|--|--RDBMS  
--|--|--|--|--|--mesg  
--oradata  
--Program Files  
--Common Files  
--|--microsoft shared  
--|--|--Filters  
--|--|--ink  
--|--|--|--ar-SA  
--|--|--|--bg-BG  
--|--|--|--cs-CZ  
--|--|--|--da-DK
```

7. Rename a File

Task:

Rename a file in a directory using the ren command.

Commands to Use:

ren file.txt renamedfile.txt

```
C:\Users\Ayaan>ren 2023-bsai-011.pptx espp.pptx
```

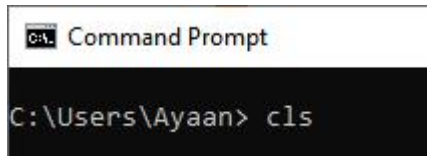
8. Clear Command Prompt Screen

Task:

Use the cls command to clear the screen of the Command Prompt.

Commands to Use:

cls



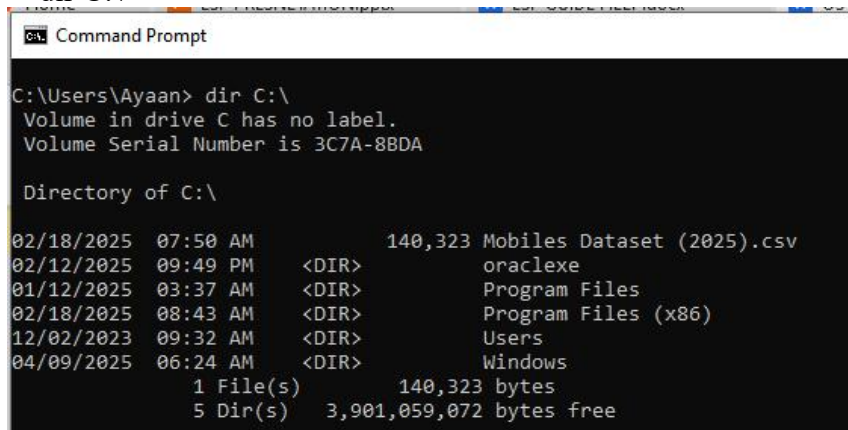
9. Display All Available Drives

Task:

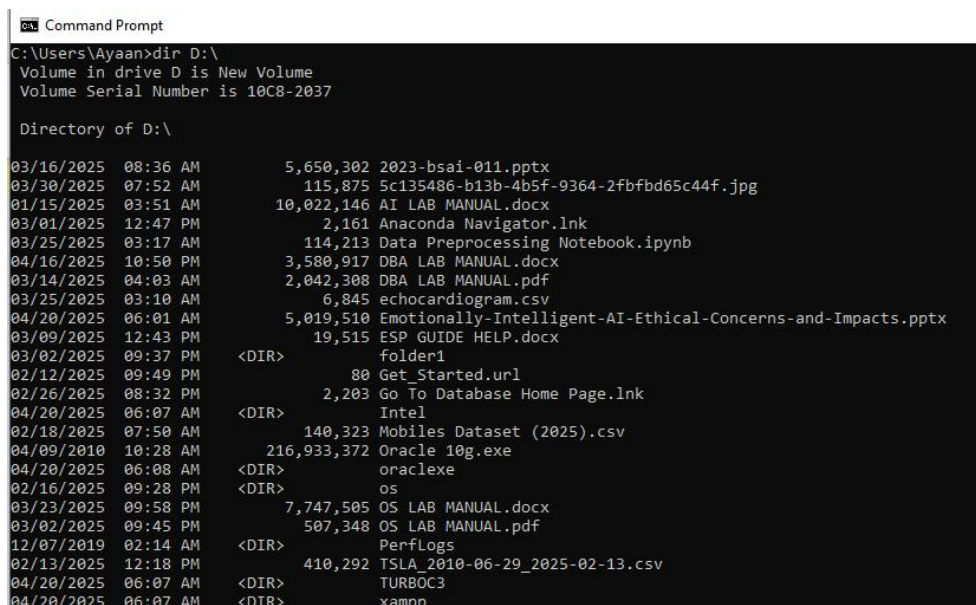
Use dir on different drives to view available files and directories in each.

Commands to Use:

dir C:\



dir D:\



10. Use Echo to Create a File with Multiple Lines

Task:

Use echo to create a text file and add multiple lines of text.

Commands to Use:

```
echo Line 1 > file.txt  
echo Line 2 >> file.txt  
echo Line 3 >> file.txt
```

11. Modify System Time

Task:

Use the time command to view and modify the system time.

Commands to Use:

```
time (view time)  
To set the time (e.g., 3:30 PM):  
time 15:30
```

```
C:\Users\Ayaan>time  
The current time is: 22:29:46.49  
Enter the new time: 15:30:30.01
```

12. Run a Program in Background

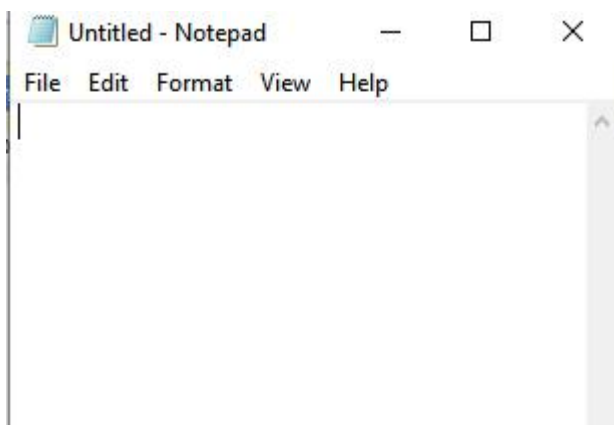
Task:

Use start to open Notepad in the background without affecting the Command Prompt.

Commands to Use:

```
start notepad
```

```
C:\Users\Ayaan>start notepad
```



LAB NO 8

FIRST COME, FIRST SERVE

I

Question 1:

n First Come First Serve (FCFS) scheduling, if the processes P1, P2, and P3 arrive in order with burst times 5 ms, 3 ms, and 8 ms respectively, what will be their average waiting time?

Given:

- P1: Burst Time = 5 ms
- P2: Burst Time = 3 ms
- P3: Burst Time = 8 ms

Since it's FCFS, the order is $P1 \rightarrow P2 \rightarrow P3$.

Waiting Time (WT):

- $P1 = 0$ ms (first process, no waiting)
- $P2 = \text{WT of } P1 + \text{Burst Time of } P1 = 0 + 5 = 5$ ms
- $P3 = \text{WT of } P2 + \text{Burst Time of } P2 = 5 + 3 = 8$ ms

Now, Average Waiting Time = $(0 + 5 + 8) / 3$
 $= 13 / 3$
 $= 4.33$ ms

Answer: Average Waiting Time = 4.33 ms

Question 2:

Four processes arrive in the order P1, P2, P3, P4 with burst times 2 ms, 4 ms, 6 ms, and 8 ms respectively. Using First Come First Serve (FCFS) scheduling, find the average turnaround time.

Given:

- P1: Burst Time = 2 ms
- P2: Burst Time = 4 ms
- P3: Burst Time = 6 ms
- P4: Burst Time = 8 ms

FCFS Order: $P1 \rightarrow P2 \rightarrow P3 \rightarrow P4$

Step 1: Completion Times (CT)

- $P1: CT = 0 + 2 = 2$ ms
- $P2: CT = 2 + 4 = 6$ ms
- $P3: CT = 6 + 6 = 12$ ms
- $P4: CT = 12 + 8 = 20$ ms

Step 2: Turnaround Time (TAT)

Turnaround Time = Completion Time - Arrival Time (Arrival Time = 0 for all)

Thus:

- P1 TAT = 2 - 0 = 2 ms
- P2 TAT = 6 - 0 = 6 ms
- P3 TAT = 12 - 0 = 12 ms
- P4 TAT = 20 - 0 = 20 ms

Step 3: Average Turnaround Time

Average TAT = (2 + 6 + 12 + 20) / 4

= 40 / 4

= 10 ms

Answer: Average Turnaround Time = 10 ms

Program no 1:

Write a C++ program to take the number of processes and their burst times (how long each task takes). Use FCFS scheduling to find the waiting time and turnaround time for each process.

Then, print the average waiting time and average turnaround time.

Key Inputs:

- Number of processes
- Burst time of each process (in order of arrival)

Key Outputs:

- Waiting time and turnaround time for each process
- Average waiting time and turnaround time

Solution:

```
#include <iostream>
using namespace std;

int main() {
    int n;

    // Take number of processes as input
    cout << "Enter the number of processes: ";
    cin >> n;

    int burstTime[100], waitingTime[100], turnaroundTime[100];
    int totalWaitingTime = 0, totalTurnaroundTime = 0;

    // Input burst times
```

```

for (int i = 0; i < n; i++) {
    cout << "Enter burst time for Process " << i + 1 << ": ";
    cin >> burstTime[i];
}

// Calculate waiting time for each process
waitingTime[0] = 0; // First process doesn't have any waiting time

for (int i = 1; i < n; i++) {
    waitingTime[i] = waitingTime[i - 1] + burstTime[i - 1];
}

// Calculate turnaround time for each process
for (int i = 0; i < n; i++) {
    turnaroundTime[i] = waitingTime[i] + burstTime[i];
}

// Calculate total waiting time and total turnaround time
for (int i = 0; i < n; i++) {
    totalWaitingTime += waitingTime[i];
    totalTurnaroundTime += turnaroundTime[i];
}

// Output waiting time and turnaround time for each process
cout << "\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n";
for (int i = 0; i < n; i++) {
    cout << "P" << i + 1 << "\t" << burstTime[i] << "\t\t"
        << waitingTime[i] << "\t\t" << turnaroundTime[i] << "\n";
}

// Output average waiting time and average turnaround time
cout << "\nAverage Waiting Time: " << (float)totalWaitingTime / n;
cout << "\nAverage Turnaround Time: " << (float)totalTurnaroundTime / n << endl;
return 0;
}

```

OUTPUT:

```

Enter the number of processes: 4
Enter burst time for Process 1: 5
Enter burst time for Process 2: 3
Enter burst time for Process 3: 8
Enter burst time for Process 4: 6

Process Burst Time  Waiting Time  Turnaround Time
P1  5      0      5
P2  3      5      8
P3  8      8     16
P4  6     16     22

Average Waiting Time: 7.25
Average Turnaround Time: 12.75

```

Program no 2:

Write a C++ program that reads the number of processes and their burst times. Based on FCFS or SJF, arrange the processes and print a simple Gantt chart.

Key Inputs:

- Number of processes
- Burst times
- Scheduling method (FCFS)

Key Outputs:

- A line of process names in execution order
- The finish time under each one (Gantt chart style)

Solution:

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

```

// Function to display Gantt chart

```

void printGanttChart(const vector<int> &processes, const vector<int> &finishTimes) {
    // Print the Gantt chart header
    cout << "\nGantt Chart: \n";

```

// Print process names in execution order

```

for (int i = 0; i < processes.size(); i++) {
    cout << "P" << processes[i] << "\t";
}
cout << endl;

```

// Print finish times line

```

int time = 0;
for (int i = 0; i < finishTimes.size(); i++) {

```



```

        cout << time << "\t";
        time = finishTimes[i];
    }
    cout << time << endl; // To print the last finish time
}

```

// Function to perform FCFS scheduling

```

void FCFS(int n, vector<int>& burstTime, vector<int>& processes) {
    vector<int> finishTimes(n);
    int currentTime = 0;

```

// Calculate finish times based on FCFS

```

for (int i = 0; i < n; i++) {
    currentTime += burstTime[i];
    finishTimes[i] = currentTime;
}

```

// Print the Gantt chart for FCFS

```

printGanttChart(processes, finishTimes);
}

```

// Function to perform SJF scheduling

```

void SJF(int n, vector<int>& burstTime, vector<int>& processes) {
    vector<int> finishTimes(n);
    vector<pair<int, int>> processBurstPair (n); // {process ID, burst time}

```

// Create a pair of process ID and burst time

```

for (int i = 0; i < n; i++) {
    processBurstPair[i] = {processes[i], burstTime[i]};
}

```

// Sort processes by burst time (Shortest Job First)

```

sort(processBurstPair.begin(), processBurstPair.end(), [](pair<int, int> a, pair<int, int>
b) {
    return a.second < b.second;
});

```

// Extract sorted processes and burst times

```

for (int i = 0; i < n; i++) {
    processes[i] = processBurstPair[i].first;
    burstTime[i] = processBurstPair[i].second;
}

```

```

int currentTime = 0;

```

// Calculate finish times based on SJF

```

for (int i = 0; i < n; i++) {
    currentTime += burstTime[i];
    finishTimes[i] = currentTime;
}

```

```

// Print the Gantt chart for SJF
printGanttChart(processes, finishTimes);
}

int main() {
    int n;
    string schedulingMethod;

    // Take number of processes as input
    cout << "Enter the number of processes: ";
    cin >> n;

    vector<int> burstTime(n), processes(n);

    // Input burst times for each process
    for (int i = 0; i < n; i++) {
        processes[i] = i + 1; // Process IDs (P1, P2, ...)
        cout << "Enter burst time for Process " << i + 1 << ": ";
        cin >> burstTime[i];
    }

    // Take the scheduling method as input
    cout << "Enter scheduling method (FCFS/SJF): ";
    cin >> schedulingMethod;

    // Call the appropriate scheduling function
    if (schedulingMethod == "FCFS") {
        FCFS(n, burstTime, processes);
    } else if (schedulingMethod == "SJF") {
        SJF(n, burstTime, processes);
    } else {
        cout << "Invalid scheduling method!" << endl;
    }
    return 0 }

```

Output (FCFS)

```
Enter the number of processes: 4
Enter burst time for Process 1: 6
Enter burst time for Process 2: 8
Enter burst time for Process 3: 7
Enter burst time for Process 4: 3
Enter scheduling method (FCFS/SJF): FCFS
```

Gantt Chart:

P1	P2	P3	P4	
0	6	14	21	24

Output (SJF)

```
Enter the number of processes: 4
Enter burst time for Process 1: 6
Enter burst time for Process 2: 8
Enter burst time for Process 3: 7
Enter burst time for Process 4: 3
Enter scheduling method (FCFS/SJF): SJF
```

Gantt Chart:

P4	P1	P3	P2	
0	3	9	16	24

LAB NO 9**SJF (Shortest Job First – Non-Preemptive)****Program no 1:**

Write a C++ program to take the number of processes and their burst times. Use Shortest Job First scheduling to arrange the tasks in order of smallest burst time. Then calculate and print the waiting time and turnaround time for each process. Also print the averages.

Key Inputs:

- Number of processes
- Burst time of each process

Key Outputs:

- Waiting time and turnaround time for each process (after sorting by burst time)
- Average waiting time and turnaround time

Solution:

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter the number of processes: ";
    cin >> n;

    int burstTime[100], processID[100], waitingTime[100], turnaroundTime[100];

    // Input burst times and assign process IDs
    for (int i = 0; i < n; i++) {
        cout << "Enter burst time for Process " << i + 1 << ": ";
        cin >> burstTime[i];
        processID[i] = i + 1;
    }

    // Sort processes based on burst time using simple selection sort
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (burstTime[j] < burstTime[minIndex]) {
                minIndex = j;
            }
        }
        // Swap burst time
        int temp = burstTime[i];
        burstTime[i] = burstTime[minIndex];
```

```

burstTime[minIndex] = temp;

// Swap process ID to keep track of the original process
temp = processID[i];
processID[i] = processID[minIndex];
processID[minIndex] = temp;
}

// Calculate waiting time and turnaround time
waitingTime[0] = 0;
turnaroundTime[0] = burstTime[0];
float totalWaitingTime = 0, totalTurnaroundTime = turnaroundTime[0];

for (int i = 1; i < n; i++) {
    waitingTime[i] = waitingTime[i - 1] + burstTime[i - 1];
    turnaroundTime[i] = waitingTime[i] + burstTime[i];
    totalWaitingTime += waitingTime[i];
    totalTurnaroundTime += turnaroundTime[i];
}

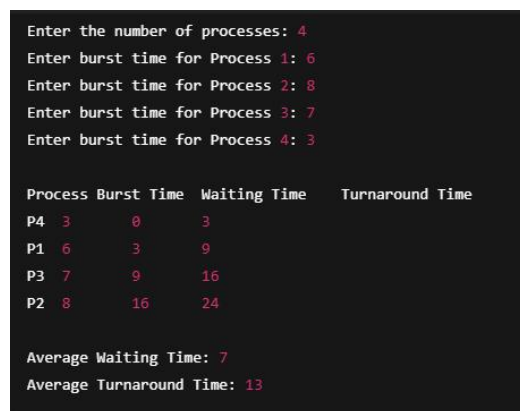
// Output results
cout << "\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n";
for (int i = 0; i < n; i++) {
    cout << "P" << processID[i] << "\t" << burstTime[i] << "\t\t" << waitingTime[i] <<
"\t\t" << turnaroundTime[i] << "\n";
}

cout << "\nAverage Waiting Time: " << totalWaitingTime / n;
cout << "\nAverage Turnaround Time: " << totalTurnaroundTime / n << endl;

return 0;
}

```

OUTPUT



```

Enter the number of processes: 4
Enter burst time for Process 1: 6
Enter burst time for Process 2: 8
Enter burst time for Process 3: 7
Enter burst time for Process 4: 3

Process Burst Time  Waiting Time  Turnaround Time
P4  3      0      3
P1  6      3      9
P3  7      9     16
P2  8     16     24

Average Waiting Time: 7
Average Turnaround Time: 13

```

Program no 2:

Problem Statement:

Write a C++ program that takes the number of processes, their arrival times, and burst times.

Use non-preemptive Shortest Job First scheduling to find the waiting time and turnaround time for each process. Then print the average waiting and turnaround times.

Key Inputs:

- Arrival time (when the process enters the system)
- Burst time (how long it takes)

Key Outputs:

- Schedule the job with the shortest burst time among the processes that have arrived
- Waiting time and turnaround time for each process
- Averages

Solution:

```
#include <iostream>
using namespace std;
```

```
int main() {
    int n;
    cout << "Enter the number of processes: ";
    cin >> n;

    int pid[100], arrival[100], burst[100];
    int waiting[100] = {0}, turnaround[100] = {0};
    int completed[100] = {0};

    // Input process details
    for (int i = 0; i < n; i++) {
        pid[i] = i + 1;
        cout << "Enter arrival time for Process " << i + 1 << ": ";
        cin >> arrival[i];
        cout << "Enter burst time for Process " << i + 1 << ": ";
        cin >> burst[i];
    }

    int time = 0, completedCount = 0;
    float totalWaiting = 0, totalTurnaround = 0;

    while (completedCount < n) {
        int idx = -1;
        int minBurst = 99999;

        // Find the shortest job among the arrived processes
        for (int i = 0; i < n; i++) {
            if (arrival[i] <= time && completed[i] == 0 && burst[i] < minBurst) {
                minBurst = burst[i];
                idx = i;
            }
        }
    }
}
```

```

    }

    if (idx != -1) {
        waiting[idx] = time - arrival[idx];
        if (waiting[idx] < 0) waiting[idx] = 0;

        time += burst[idx];
        turnaround[idx] = time - arrival[idx];

        completed[idx] = 1;
        completedCount++;

        totalWaiting += waiting[idx];
        totalTurnaround += turnaround[idx];
    } else {
        time++; // No process is ready, advance time
    }
}

// Output
cout << "\nProcess\tArrival\tBurst\tWaiting\tTurnaround\n";
for (int i = 0; i < n; i++) {
    cout << "P" << pid[i] << "\t" << arrival[i] << "\t" << burst[i] << "\t"
        << waiting[i] << "\t" << turnaround[i] << endl;
}

cout << "\nAverage Waiting Time: " << totalWaiting / n;
cout << "\nAverage Turnaround Time: " << totalTurnaround / n << endl;

return 0;
}

```

OUTPUT:

```

Enter the number of processes: 4
Enter arrival time for Process 1: 0
Enter burst time for Process 1: 7
Enter arrival time for Process 2: 2
Enter burst time for Process 2: 5
Enter arrival time for Process 3: 4
Enter burst time for Process 3: 1
Enter arrival time for Process 4: 5
Enter burst time for Process 4: 2

Process Arrival Burst  Waiting Turnaround
P1 0 7 0 7
P2 2 5 6 11
P3 4 1 3 4
P4 5 2 7 9

Average Waiting Time: 4
Average Turnaround Time: 7.75
    
```

Program no 3:

Problem Statement:

Write a C++ program that reads the number of processes and their burst times. Based on SJF,

arrange the processes and print a simple Gantt chart.

Key Inputs:

- Number of processes
- Burst times
- Scheduling method (SJF)

Key Outputs:

- A line of process names in execution order
- The finish time under each one (Gantt chart style)

Solution:

```
#include <iostream>
```

```
#include <vector>
```

```
#include <algorithm>
```

```
using namespace std;
```

```
// Function to display Gantt chart
```

```
void printGanttChart(vector<int> &processes, vector<int> &finishTimes) {
```

```
    // Print the Gantt chart header
```

```
    cout << "\nGantt Chart: \n";
```

```
    // Print process IDs
```

```
    for (int i = 0; i < processes.size(); i++) {
```

```
        cout << "P" << processes[i] << "\t";
```

```
    }
```



```

cout << endl;

// Print the finish times line
int time = 0;
for (int i = 0; i < finishTimes.size(); i++) {
    cout << time << "\t";
    time = finishTimes[i];
}
cout << time << endl; // To print the last finish time
}

int main() {
    int n;

    // Read number of processes
    cout << "Enter the number of processes: ";
    cin >> n;

    vector<int> burstTime(n), processes(n), finishTime(n);
    int totalTime = 0;

    // Read burst times for each process
    for (int i = 0; i < n; i++) {
        processes[i] = i + 1; // Process IDs
        cout << "Enter burst time for Process " << i + 1 << ": ";
        cin >> burstTime[i];
        totalTime += burstTime[i];
    }

    // Sort the processes based on burst time (Shortest Job First)
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (burstTime[i] > burstTime[j]) {
                // Swap burst time
                swap(burstTime[i], burstTime[j]);
                // Swap process IDs
                swap(processes[i], processes[j]);
            }
        }
    }

    // Calculate the finish times for each process
    int currentTime = 0;
    for (int i = 0; i < n; i++) {
        currentTime += burstTime[i];
        finishTime[i] = currentTime;
    }

    // Print the Gantt Chart and process finish times

```

```
printGanttChart(processes, finishTime);  
  
return 0;  
}
```

OUTPUT:

```
Enter the number of processes: 4  
Enter burst time for Process 1: 6  
Enter burst time for Process 2: 8  
Enter burst time for Process 3: 7  
Enter burst time for Process 4: 3  
  
Gantt Chart:  
P4  P1  P3  P2  
0   3   9  16  24
```